Reliable API Sync

1. What was built and why

A Python integration reads a paginated product API, stores the latest validated product state in SQLite and queues notifications when prices or currencies change. It addresses two common integration problems: retrying a request can duplicate a notification, and an update can be lost if product state is saved separately from the corresponding notification task.

The implementation uses a transactional outbox: the changed product and its pending event are stored in the same database transaction. Delivery is a separate operation with a stable event identifier, bounded retries and explicit acknowledgement. This is an integration foundation, not a deployed price-monitoring website, a marketplace scraper or a continuously running monitoring service.

2. API contract and state

GET /products returns items and next_cursor. Every item contains exactly id, revision, price and currency. IDs are restricted strings, revisions are positive integers, prices are nonnegative decimal strings with at most two fractional digits, and allowed currencies are USD, GBP, CNY and AED. Subsequent pages use the returned cursor.

Stored information	Purpose
products: ID, revision, price, currency	Latest accepted state per product
outbox: event ID, JSON payload, delivered flag	Pending and acknowledged notifications
event ID: SHA-256 of ID + ':' + revision	Stable identity for retries of one revision

Baseline observations create product records but no change events. A higher revision updates state and creates an event only if price or currency changed. Older revisions are ignored. An equal revision with different values is rejected, because revision reuse violates the source contract. This assumes one source/feed per database and consistent revision semantics.

3. Processing and delivery method

Fetch and validate all pages before starting the database transaction. Identical records repeated across pages are collapsed; conflicting duplicates fail the fetch. The Store interface independently rejects a list containing duplicate IDs. Pagination loops, response-size limits and excessive item/page counts are checked.

The sender POSTs each pending event with an Idempotency-Key equal to its event ID. It marks an event delivered only after receiving accepted=true and the matching event_id. The receiver must durably deduplicate that key before applying side effects. The mechanism provides at-least-once delivery attempts; it does not make every receiver exactly-once automatically.

4. Synthetic data and failure injection

demo.py starts one HTTP server on 127.0.0.1 using a temporary port. It serves both the source API and the receiver; no external store, customer account or messaging service is contacted. It generates 12 products, DEMO-01 through DEMO-12, with initial USD prices 101 through 112. Pages contain 5, 5 and 2 records.

The first fetch deliberately returns HTTP 429. After baseline storage, the SQLite connection is closed and reopened in the same Python process. The first three products advance to revision 2 and decrease by USD 5: 101 to 96, 102 to 97 and 103 to 98. The other nine products remain unchanged. The same revised feed is then replayed.

For the first notification, the receiver stores the event but deliberately responds HTTP 503, simulating a failed acknowledgement after processing. The sender retries the same event ID, and the receiver's in-memory dictionary suppresses a second unique event. After delivery, the database is closed and reopened again before another delivery attempt.

5. Expected versus observed results

Stage	Acceptance criterion	Observed
Baseline after one HTTP 429	Store all products; emit no changes	12 products, 0 events
Revised feed	One event for each changed product	3 updates, 3 events
Same revision replay	No additional event creation	0 new events
First acknowledgement fails with 503	Retry using the same event identity	4 POST attempts, 3 unique events
Reload SQLite and deliver again	Acknowledged events stay acknowledged	0 deliveries, 0 pending events

Twenty automated test methods passed under Python 3.12.14 on macOS. Nineteen use in-memory state or controlled fake clients/openers; one runs the real loopback HTTP scenario. They cover schema/decimal/revision validation, transactions, pagination, conflict handling, replay, bounded retry exhaustion, non-retryable HTTP 401, invalid acknowledgement and the end-to-end fixture.

Important distinction: reopening SQLite demonstrates persisted-state reuse, not a separate operating-system process restart or a crash/power-loss experiment. The receiver does not restart during this test and its deduplication set is not durable. No production uptime, throughput, latency or long-duration fault-injection result is claimed.

6. Reproduce and inspect

Python 3.10+ is declared; the recorded run used Python 3.12.14 and standard-library modules only. The demo/test needs permission to bind a loopback port. Run from the repository directory with a fresh output directory:

```
python -m unittest -v
python demo.py --output demo-output
```

Inspect demo-output/evidence.json for stage counts, the actual GET/POST status trace, and the three event payloads. SQLite state is local and is not included in the public sample. sample_evidence.json preserves the recorded synthetic run without credentials or live URLs.

For an approved real API with this contract, --base-url and --db enable synchronization; notifications require the separate explicit --send-to option. A source bearer token can be read from a named environment variable. The CLI constructs a separate receiver client so the source token is not forwarded to the destination. Receiver authentication remains to be implemented for the target provider.

Evidence	Location
HTTP client, validation, state and outbox	api_sync.py
Product fixture, local server and injected failures	demo.py
Unit and loopback integration checks	test_api_sync.py
Actual stage counts and HTTP trace	sample_evidence.json
Recorded test run and source hashes	verification.json

7. Limits and production acceptance

The system requires a trustworthy, monotonic revision contract and a receiver with durable idempotency. It is single-worker and lacks distributed locking, production receiver authentication, provider-specific adapters, scheduled operation, monitoring and full-refresh deletion handling. HTTPS is required except for explicit loopback URLs; redirects are disabled. Default requests use three attempts and five-second timeouts; Retry-After accepts bounded delta seconds, not HTTP-date values.

An event ID alone does not prevent duplicate external side effects. A production receiver needs an atomic transaction joining its deduplication record to its business action, or an equivalent provider guarantee. Separate-process restarts, crashes between network and database operations, receiver restarts and concurrent workers require additional experiments before stronger reliability claims are made.

Before client delivery, agree on feed identity, schema, change semantics, access permission, rate limits and receiver behavior. Then implement the authorized adapter and authentication, test failures in the actual target environment, and define operational responsibility. This demo contains no scraping, account login automation or external notification delivery.

8. Contribution and evidence boundary

This independent engineering project covers implementation, testing and documentation. All products and faults are synthetic. It demonstrates explicit integration contracts and testable failure handling, not paid client outcomes or an already deployed monitoring service.